Design & Analysis of Algorithms

Page
Date

2 types of complexity
- How space calculated
- Name of notation used
- 3 notations - name & symbol
- Graph Big oh (O)
- Relation $f(n) \rightarrow$ & g
- $f(n) = 3n+2$ $g(n) = n$ — Device
- In above case why we not take $g(n) = n^2, n^3$ etc even if $f(n) = O\,g(n)$ for them as well
- Big omega graph
- Mathematics relation
- $f(n) = 3n+2$ $g(n) = n$ → Device
- Big theta graph
- Mathematical relation
- $f(n) = 3n+2$ $g(n) = n$ → Device
- O →
- Ω →
- Θ →
- which mostly used of 3 (why)
- linear search Ω, O, Θ
- 2 type of Algo
- Relation B/w both
- Time complexity if no iteration no recursion
- Find time complexities of

a)
```
A()
{
i=1, s=1;
while(s<=n)
{
i++;
s=s+i;
printf("\n");
}
}
```

Scanned with CamScanner

b) A(){
int i, j, k, n;
for(i=1; i<=n; i++){
for(j=1; j<=i; j++){
for(k=1; k<=100; k++)
{ printf("ravi"); }
}
}

c) A(){
int i, j, k, n;
for(i=1; i<=n; i++){
for(j=1; j<=i^2; j++){
for(k=1; k<=n/2; k++){
printf("Ravi");
}}}

d) A(){
for(i=1; i<n; i=(i*2)
printf("ravi");
}

e) A(){
int i, j, k;
for(i=n/2; i<=n; i++)
for(j=1; j<=n/2; j++)
for(k=1; k<=n; k=k*2)
printf("ravi");
}

f) A(){
int i, j, k;
for(i=n/2; i<=n; i++)
for(j=1; j<=n; j=2*j)
for(k=1; k<=n; k=k*2)
printf("ravi");
}

Scanned with CamScanner

g) A() {
while (n>1) {
n = n/2;
}
}

h) A() {
for (i=1; i<=n; i++)
for (j=1; j<=n; j=j+i)
printf("ravi");
}

i) A() {
int n = 2^k;
for (i=1; i<=n; i++) {
j=2;
while (j<=n) {
j = j^2;
printf("ravi");
}
}

Q How recursive decided by anchor base induction
Q find for.
A(n) {
if (n>1)
return (A(n-1))
}

Q $T(n) = n + T(n-1)$; $n>1$
$= 1$; $n=1$

Q What is recursion tree method
Q Draw // (for $T(n) = 2T(n/2) + n$)

// // // , $T(n) = 2T(\frac{n}{2}) + n$; $n>1$
$= 1$; $n=1$

Scanned with CamScanner

Page 150
Date

Master theorem - how to decide b/w 2^n, n^2

2 ways

Solve for 2^n, n^2

Solve for n^2, $n\log n$

" " " n, $(\log n)!^{00}$

$n^{\log n}$, $n\log n$

$n^{\sqrt{\log n}}$, $\log\log n$

$f(n) = \begin{cases} n^3 & 0 < n < 10,000 \\ n^2 & n \geq 10000 \end{cases}$

$g(n) = \begin{cases} n & 0 < n < 100 \\ n^3 & n > 100 \end{cases}$ Find relation

Relation b/w $f_1 = 2^n$, $f_2 = n^{3/2}$, $f_3 = n\log n$, $f_4 = n^{\log n}$

Diff b/w $(\log n)^2$ & $\log^2 n$

$T(n)$ = General for master thm

3 conditions & all subcondition - master thm

Apply Master theorem on

a) $T(n) = 3T(\frac{n}{2}) + n^2$

b) $T(n) = 4T(\frac{n}{2}) + n^2$

c) $T(n) = T(\frac{n}{2}) + n^2$

d) $T(n) = 2^n T(\frac{n}{2}) + n^n$

e) $T(n) = 16T(\frac{n}{4}) + n$

f) $T(n) = 2T(\frac{n}{2}) + n\log n$

g) $T(n) = 2T(\frac{n}{2}) + \frac{n}{\log n}$

h) $T(n) = 2T(\frac{n}{4}) + n^{0.51}$

i) $T(n) = 0.5T(n/2) + 1/n$

j) $T(n) = 6T(n/3) + n^2\log n$

k) $T(n) = 64T(n/8) - n^2\log n$

l) $T(n) = 7T(n/3) + n^2$

m) $T(n) = 4T(n/2) + \log n$

n) $T(n) = \sqrt{2}T(n/2) + \log n$

o) $T(n) = 2T(n/2) + \sqrt{5}$

p) $T(n) = 3T(n/2) + n$

q) $T(n) = 3T(n/3) + \sqrt{n}$

Scanned with CamScanner

a) $T(n) = 4T(\frac{n}{2}) + C(n)$

Q. Space complexity (Case of T(n))

Q. Space complexity for

a)
```
Algo(A, 1, n){
    int i;
    create B[i];
    for(i=1 to n)
        B[i] = A[i];
}
```

b)
```
Algo(A, 1, n){
    int i;
    for(i=1 to n)
        A[i] = 0;
}
```

c)
```
Algo(A, 1, n){
    create B[n,n];
    int i,j;
    for(i=1 to n)
        for(j=1 to n)
            B[i,j] = A[i];
}
```

d)
```
A(n){
    if(n>=1){
        A(n-1);
        printf(n);
    }
}
```

e)
```
A(n){
    if(n>=1){
        A(n-1);
        printf(n);
        A(n-1);
    }
}
```

Q. 3 types of recursion ⟵

Q. 2 ways to get space complexity in

Q. Type of recursion in (d)

Q. Time complexity for (d)

Q. (e) by both stack & tree

Q. Conclusion from tree (A(2) repeated)

Q. Solution

Q. Time complexity for (e)

Scanned with CamScanner

25/9/20 Pg- 226

Page 152
Date

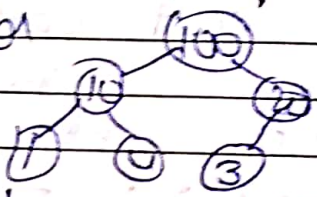
- Amortized analysis – intro costly operation at most
- Defn & what is average called –
- 2 points
- Where used
- 3 technique in amortized analysis
- Aggregate Analysis – 1 Relation
- Hash table insertion – 3 steps
- Details of 1 to 10
- Amortized cost – Calculate
- result
- Bubble sort – code
- Time complexity – worst
- best
- Note
- Bucket sort – where used
- Visualise as hashing & sort 0.35, 0.12, 0.43, 0.15, 0.04, 0.50, 0.13, 0.2.
- Best Case, worst case
- How decided to put new # if all already there
- Counting sort assumption
- Sort 2 2 3 4 1 5 1 5
- ↓ Visualise
- Time & Space complexity
- 2 disadvantages
- Insert, search, find min, Delete min for
  a) unsorted array
  b) sorted array
  c) linked list
- Observation from above Q.
- → For min heap
- What is heap
- Rules for binary.

Scanned with CamScanner

Page
Date

Tell, heap

- Max heap
- Min heap
- How heap looks like (DS) & what we actually use
- left child, right child
- Parent
- Right child is $(i+1) * 2$ / left shift
- Work array for



why

- Now, even if tree necessarily almost complete
- $L(i)$, $R(i)$ are — right $R(i)$'s — left
- Tell if max heap. If not tell which point it is not heap

a) 25, 12, 16, 13, 10, 8, 14
b) 25, 14, 16, 13, 10, 8, 12
c) 25, 14, 13, 16, 10, 8, 12
d) 25, 14, 12, 13, 10, 8, 16
e) 14, 13, 12, 10, 8
f) 14, 12, 13, 8, 10
g) 14, 13, 8, 12, 10
h) 14, 13, 12, 8, 10
i) 89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70

- 14, 13, 12, 10, 8 — heap
- 8, 10, 12, 13, 14 — heap
- Conclusion
- Time complexity total
- Why not sorted

end of new

Scanned with CamScanner

Height of complete binary tree edges (root)

Table of Height V/s node

Q- Largest node for height h

Q- " height for node n

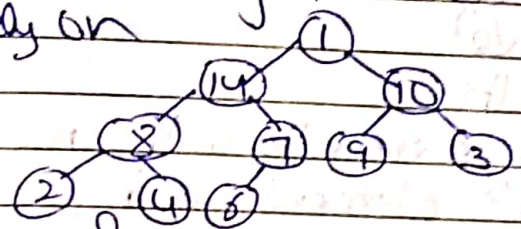
Q- $\lfloor \log_2 n \rfloor$ — if not integral then

Q- for ternary

Q- If complete/almost/ binary tree then formula to get leaf nodes partition

Q- Code — Array to max heap

Q- Apply on

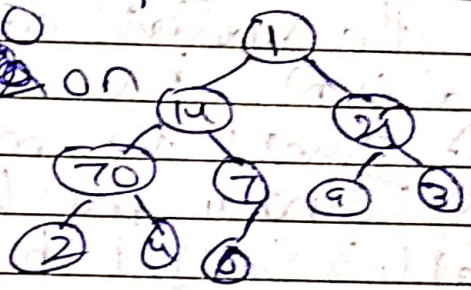


Q- Time & Space complexity

~~Q- Give exception to above pic~~

Q- Build heap (A) — algo

Q- eg of Build heap on



Q- Time complexity — wrong way

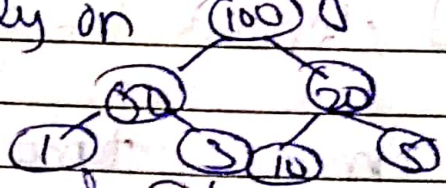
Q- " " " " — Recurr

— Result

Q- Space "

Q- Delete Max from Max heap algo

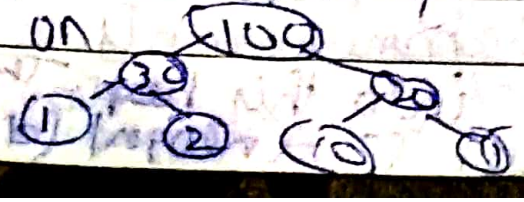
Q- Apply on



Q- Time & Space complexity

Q- Algo ↑ value of node

Q- Apply on

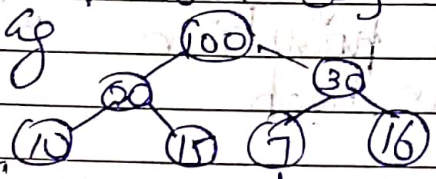


key = 300

i = 5

Scanned with CamScanner

Time complexity
Space "
Decreasing a key should all —
Time →
Insert an element — 2 ways
Max heap time of find Max, Delete Max, insert ↑, ↓, find Min, delete random, search random
heap sort — algo:
eg



Time complexity
In a heap with n elements with smallest element at root, the 7th element can be found in time
a) $\Theta(n \log n)$ (b) $\Theta(n)$ (c) $\Theta(\log n)$ (d) $\Theta(1)$
In a binary max heap containing n elements, the smallest element can be found in time
a) $\Theta(n)$ (b) $\Theta(\log n)$ (c) $\Theta(\log \log n)$ (d) $\Theta(1)$
Insert into max heap 35, 15, 20, 30, 12, 25, 16
Ternary max heap is represented by
a) 1, 3, 5, 6, 8, 9 (b) 9, 6, 3, 1, 8, 5 (c) 9, 3, 6, 8, 5, 1
(d) 9, 5, 6, 8, 3, 1 & insert 7, 2, 10, 4
Consider process of inserting an element into a max heap. If we perform a binary search on the path from new leaf to root to find the position of newly inserted element, the # of comparisons performed are —
We can insert in — time only
→ Same question from insertion sort —
We have a binary heap on 'n' elements and wish to insert 'n' more elements (not necessarily one after another) into this heap. The total time required for this is
a) $\Theta(\log n)$ (b) $\Theta(n)$ (c) $\Theta(n \log n)$ (d) $\Theta(n^2)$

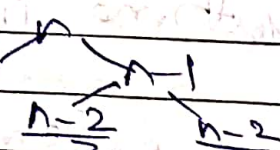
Scanned with CamScanner

Page 16
Date

Insertion Sort - Algo
Trace 9, 6, 5, 0, 8, 2, 7, 1, 3
Worst case
Best "
Space complexity
↓ algo
what if we replace linear by binary search in insertion sort
What if we use doubly LL in place of array
Imp Conclusion from above 2
Merge Sort (Better than ___ sort)
Merging - meaning
Eg of 1, 5, 7, 8, 2, 4, 6, 9 (Merging)
Algo for MERGE
space Complexity
Why added at end then 10, 20, 30, 40, 5, 6, 9
Category of Merge Sort
Merge Sort - Algo
Apply on [9|6|5|0|8|2]
Draw table (tree)
Merge 2 ___ lists
Merge Sort 2 ___ lists
Space Complexity - how decided
" " - Value
Merge sort time complexity - master theorem
" " - Value
Given n elements, merge them into one sorted list using merge procedure (Find out the)
Given k sorted lists each of size n, what is total time required to merge them into one single list
n strings each of length n are given then what is time taken to sort them using 2 way merge

Scanned with CamScanner

Page
Date

Merge Sort Uses a) DAC b) Greedy c) Backtracking
n & n time time taken
find after 2 Passes 20, 47, 15, 8, 9, 4, 40, 30, 12, 17
Quick sort & Merge sort follow —
Merge sort is to merge then quick sort is to
Write code of Partition
Pivot assumed at —
Best
Time Complexity (Why)
Apply on 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11
Quicksort algo
Apply & make tree on 51, 73, 62, 15, 37, 26, 45
Space complexity — avg
" " = Worst case
Write recursive eqn
Time complexity
Worst Case
Eqn
Quicksort on 1, 2, 3, 4, 5, 6
Time Complexity & eqn
Quicksort on 2, 2, 2, 2, 2, 2
Time complexity & eqn
Best Case
Apply on prob 18, 9 (Text)
Time Complexity
" " for


" " for O(n-1) → Generalize
The median of n elements can be found in O(n) time. which one of the following is true about complexity of quicksort, in which median is selected as pivot?

Scanned with CamScanner

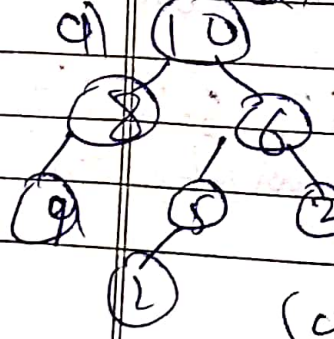
Page 16
Date

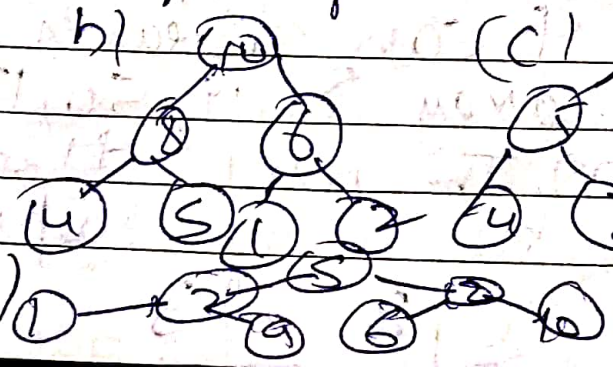
Q- In quicksort, for sorting 'n' elements the (n/4)th smallest element is selected as pivot using O(n) time algorithm. What is worst case time complexity of quicksort
Q) 1, 2, 3, 4, ---- n — T_1 Relation b/w
n, n-1, n-2, ---- 3, 2, 1 — T_2 T_1 & T_2
Q: Time complexity for $\frac{n}{5}$, $\frac{4n}{5}$ split
Q: Radix sort apply it on 804
26
5
64
52
7

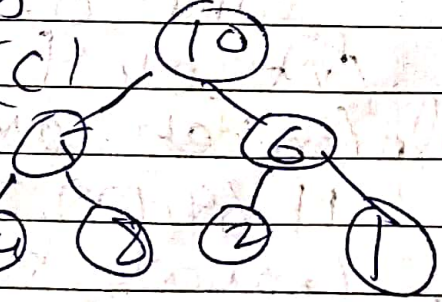
Q- Radix sort algo
Q: which sort preferred
Q: Unity
Q: Time complexity
Q: what is b, l
Q: Time complexity if k very large
Q: Space complexity
Q: Best Case

Note 2016 Q. The worst case running times of insertion sort, merge sort, quicksort are
a) $\Theta(n \log n)$, $\Theta(n \log n)$, $\Theta(n^2)$
b) $\Theta(n^2)$, $\Theta(n^2)$, $\Theta(n \log n)$
c) $\Theta(n^2)$, $\Theta(n \log n)$, $\Theta(n \log n)$
d) $\Theta(n^2)$, $\Theta(n \log n)$, $\Theta(n^2)$

Note Q Take of Algo, Best, Avg, worst Time, worst Space (except Tim & Shell sort)
Criteria

Q- Max heap from following is
a)


b)


c)


Scanned with CamScanner

Gate 2014, 2005
Q A priority queue is implemented as a max-heap and initially it has 3 elements. The level order traversal of heap is 10, 8, 5, 3, 2. Two new elements 1 & 7 are inserted into heap. The level order traversal of heap after insertion is
a) 10, 8, 7, 3, 2, 1, 5 b) 10, 8, 7, 2, 3, 5
c) 10, 8, 7, 1, 3, 1, 5 d) 10, 8, 7, 5, 3, 2, 1

Gate 2016
Q. An operation delete(i) for a binary heap DS is to be designed to delete the item in the ith node. Assume that the heap is implemented in an array & i refers to the ith index of the array. If the heap tree has depth 'd' (number of edges on path from root to farthest leaf), then what is the time complexity to refix the heap efficiently after the removal of the element
a) O(1) b) O(d) but not O(1) c) $O(2^d)$ but not O(d) d) $O(d2^d)$ but not $O(2^d)$

Gate 2010
Q. A complete binary min-heap is made by including each integer in [1, 1023] exactly once. The depth of a node in the heap is the length of the path from root of the heap to that node. Thus the root is at depth 0. The max depth at which integer 9 can appear is

Gate 2005
Q. Consider array 39, 19, 50, 17, 12, 15, 2, 5, 7, 11, 69, 100. The min # of interchanges to convert to a max heap is
a) 4 b) 5 c) 2 d) 3

From Pg 258 Gate-III copy

Gate 2015
Q. Consider a max heap represented by the array 40, 30, 20, 10, 15, 16, 17, 8, 4. Now consider that a value 35 is inserted into this heap. After insertion new heap is
a) 40, 30, 20, 10, 15, 16, 17, 8, 4, 35
b) 40, 35, 20, 10, 30, 16, 17, 8, 4, 15
c) 40, 30, 20, 10, 35, 16, 17, 8, 4, 15
d) 40, 35, 20, 18, 15, 16, 17, 8, 4, 30

Gate 2015
Q. Consider a complete binary tree where the left & right subtree of the root are max heaps. The lower bound for the no. of operations to convert the tree to a heap is.
a) $\Omega(\log n)$ b) $\Omega(n)$ c) $\Omega(n \log n)$ d) $\Omega(n^2)$

Q. Binary search & more names
Q. where applied / not applied
Q. How it works - 3
Q. Problem statement
Q. Algo of Binary Search
Q. Implementation using arrays - code
Q. Recurrence equation
Q. Time complexity
Q. General defn search algorithms
Q. DS for
Q. Another category of searching used
Q. Linear search (other name)
Q. where applied / not applied
Q. How it works?
Q. Problem Statement
Q. Algorithm
Q. Code
Q. Time complexity, Best, worst, Average

Scanned with CamScanner

- Recurrence eqn
- Implementation using LL code
- Where greedy algo used
- Name some optimization problems
- Eg diagram
- 1st way
- Time complexity
- 2nd way
- Name 2 programming paradigms
- Usefulness of greedy
- DP
- Time complexity of it
- Draw dig to show all optimization problems
- Solve Knapsack

	ob1	ob2	ob3
Profit	25	24	15
Weight	18	15	10

& find max profit if W=20

- Algorithm for greedy knapsack
- Apply on n=15

objects	1	2	3	4	5	6	7
profits	10	5	15	7	6	18	3
weight	2	3	5	7	1	4	1

- Time complexity
- What if we implement by max heap time
- Conclusion
- Solve

object	1	2	3	4	5	
P	2	28	25	18	9	n=15
W	1	4	5	3	3	

- If W same for all then what we do

Scanned with CamScanner

Solution pg-266

Q. If want to represent a file having 100 characters & only abcd as values # of bits?
Q. Is it possible to reduce # of bits in above Q?
Q. Name of method implied
Q. eg for a=50 c=5 # of bits?
b=40 d=5 . Q choice.
Q. Caution
Q. Apply lexical analyser for abcdb for huffman
Q. How to decide bits in huffman (tree)
Q. Apply & build tree & assign bits to a=50, b=40, c=5, d=5 & find # of bits per character
Q. 2 imp features of tree
Q. other name of net total bits
Q. Solve for a=40, b=30, c=20, d=5, e=3, f=2
Q. Huffman algo
Q. Apply on a=45, b=13, c=12, d=16, e=9, f=5 and find bit representation.
Q. Time complexity
Q. Space
Q. Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 which huffman code for letter a, b, c, d, e, f?
a) 0, 10, 110, 1110, 11110, 11111
b) 11, 10, 011, 010, 001, 000
c) 11, 10, 01, 001, 0001, 0000
d) 110, 100, 010, 000, 001, 111

Q. What is average length of correct answer in above question a) 3 b) 2.1875 c) 2.25 d) 1.9375
Q. Solve a=10, e=15, i=12, o=3, u=4, s=13, t=1
Q. Find Max. Profit for

J	J_2	J_3	J_4	D-Deadline of job
P	6	8	5	10

P-Profit when job done

Every job will take 1 unit of time to complete & jobs can't be done simultaneously

Scanned with CamScanner

	J_1	J_2	J_3	J_4	J_5	J_6
Q-1 P	5	3	3	2	4	2
P	200	180	190	300	120	100

Q- Time complexity

Q-

	J_1	J_2	J_3	J_4	J_5
P	2	4	3	1	10
D	3	3	3	4	4

Q-

	J_1	J_2	J_3	J_4	J_5	J_6	J_7	J_8	J_9
P	15	20	30	18	18	10	23	16	25
P	7	2	5	3	4	5	2	7	3

Q- files a b c total # of record
10 20 30 movements must be
as min as possible? How solved?

Q- Solve for ↑

Q- Time complexity

Q- Solve for a b c d e f
40 10 20 15 25 30

Q- 2 type of graph

Q- Simple graph features

Q- min edges in simple graph

Q- max

Q- ~~Time~~ Edge complexity worst case

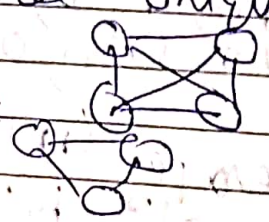
Q- Weighted graph

Q- Unweighted "

Q- Min edges to span n vertex - value

Q- in spanning tree unique

Q- Spanning tree for

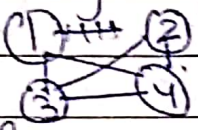
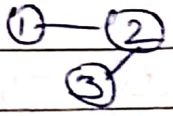
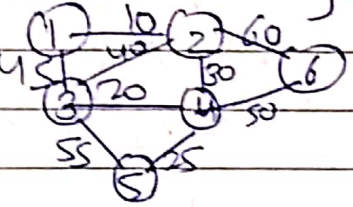
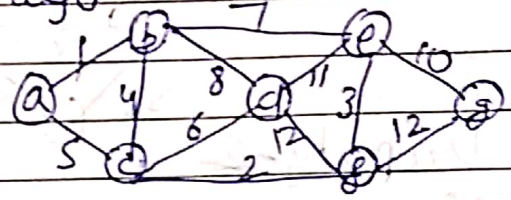
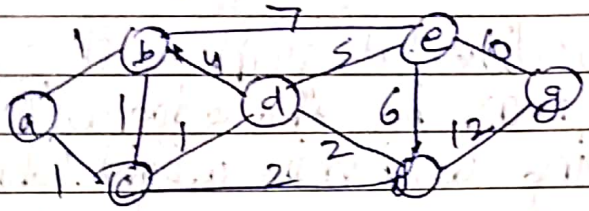


Q- formula general

Q- value only for — graph

Q- what is complete graph

Scanned with CamScanner

Q: How to find if graph not complete - theorem
Q: Whole theory of Kirchoff's theorem
Q Apply on 
Q: Apply Kirchoff on K_3
Q: 
Q: 2 greedy algo's to find min spanning tree in weighted graph
Q: Solve by prims' algo.
a) 
b) 
Q: 
Q Apply prims on (a)

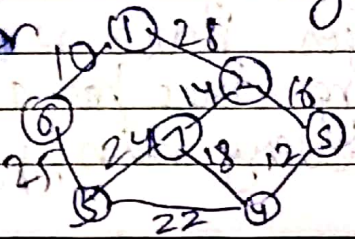
	1	2	3
1	0	10	30
2	10	0	20
3	30	20	0

(b)

	1	2	3	4
1	0	10	10	50
2	10	0	40	30
3	10	40	0	20
4	50	30	20	0

Gate 2010
(c)

	0	1	2	3	4
0	0	1	8	1	4
1	1	0	12	4	9
2	8	12	0	7	3
3	1	4	7	0	2
4	4	9	3	2	0

Q Prims algo Algorithm assumptions
Q
Q using min heap
Q Apply for 

Scanned with CamScanner

Apply min heap on prim's algorithm

Time complexity = Method 1

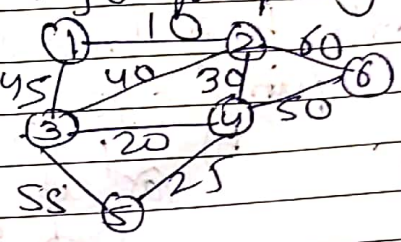
" " " " " 2

What if we use fibonacci heap

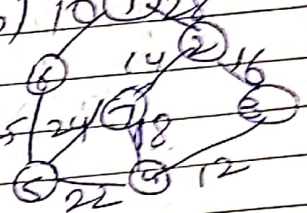
Is O(E log V) better or O(V²) - dense / sparse

Kruskal's algo - Property - 1

Apply on a)



b)



Property - 2

Gate 2000

Let 'G' be an undirected connected graph with distinct edge weights. Let max be the edge with max weight & min with min weight. Which of the following is true

a) Every min spanning tree of 'G' must contain max

b) If max is in a minimum spanning tree then its removal must disconnect 'G'

c) No min spanning tree contains max

d) 'G' has a unique min spanning tree

Gate 2007

Let 'w' be min weight among all weights in an undirected connected graph. Let 'e' be a specific edge of weight 'w'. Which of the following is FALSE

a) There is a min spanning tree containing 'e'

b) If 'e' is not in a min spanning tree 'T', then in the cycle formed by adding 'e' to 'T', all edges have same weight.

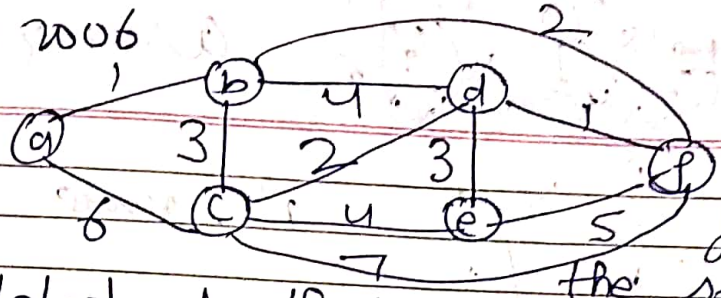
c) Every min spanning tree has an edge of wt 'w'

d) 'e' is present in every min spanning tree

Scanned with CamScanner

Gate 2006

Q-1

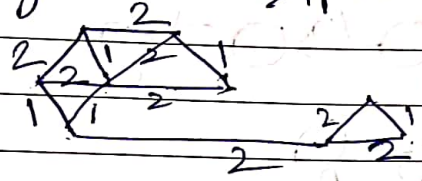


Which of the following can't be the sequence of edges added, in that order to min spanning tree using kruskal's algorithm.

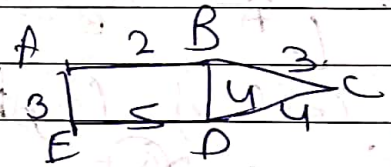
a) (a-b) (d-f) (b-f) (d-c) (d-e)
b) (a-b) (d-f) (d-c) (b-f) (d-e)
c) (d-f) (a-b) (d-c) (b-f) (d-e)
d) (d-f) (a-b) (b-f) (d-e) (d-c)

2014

Q The # of min spanning trees for weighted graph



Q-1 # of spanning trees for



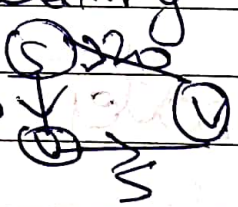
Q A complete, undirected, weighted graph 'G' is given on the vertex {0, 1, 2 ---- n-1} for any fixed 'n'. Draw the min spanning tree of G if

a) The weight of edge (u, v) is |u-v|
b) " " " " " " (u+v)

Q How shortest path & min spanning tree diff give an eg

Q What is relaxing an edge

Q Apply on

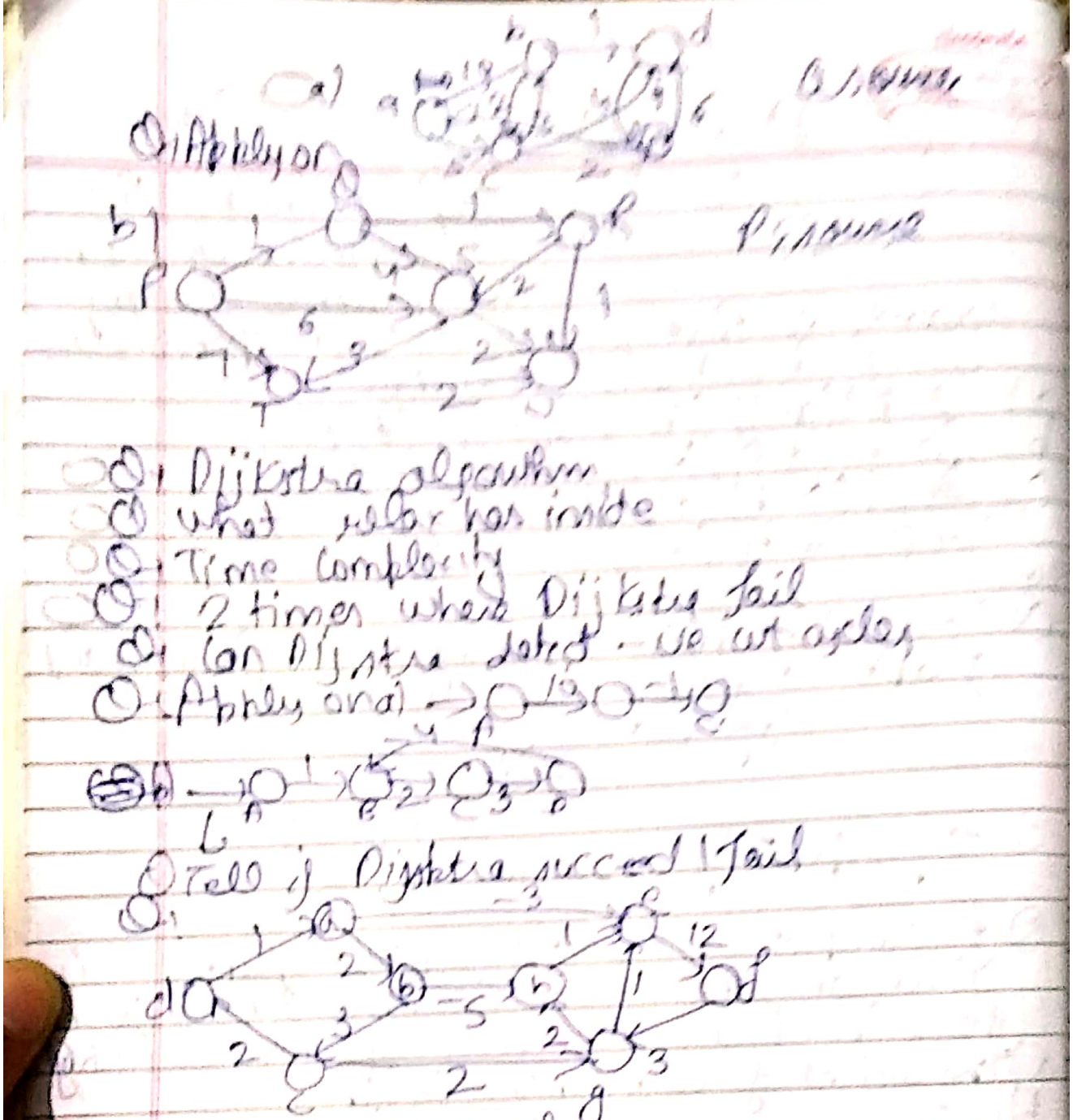


Q Must applied algorithm for shortest path

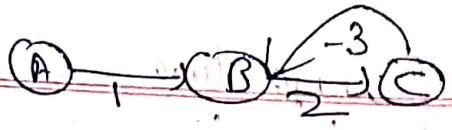
Q Dijkstra where applied (not applied

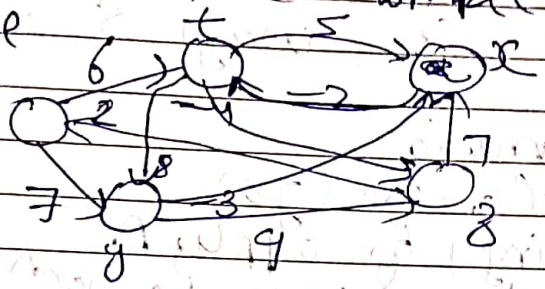
Q-1 Theory

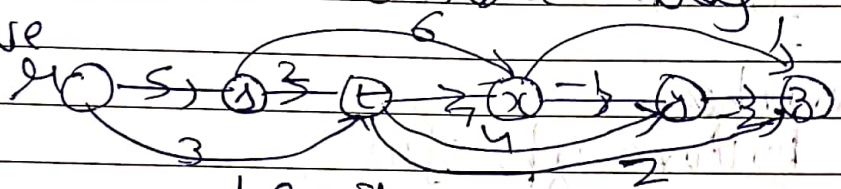
Scanned with CamScanner



- Dijkstra algorithm
- what relax has inside
- Time complexity
- 2 times when Dijkstra fail
- Can Dijkstra detect. ve wt edges
- Applies on ai
- Tell if Dijkstra succeed / fail
- Tell about sequence also
- Bellman-ford (2 types of analysis)
- Time complexity of Dijkstra
- Real life
- BF
- Max SP if n nodes
- eg of
- Q- what if still relaxing after n-1 edges

- Eg of 

- Q: Find time complexity
- Solve 

- Q- Algo
- Time Complexity
- what is DAG
- we allow — also ? why
- Procedure to solve dag
- Solve 

- Time complexity
- DAG → Algo
- DP → similar to — but not —
- Programming meaning
- Dynamic "
- DAG draw dig
- Problem (subproblem)
- Eg of Fibonacci — Draw tree etc.
- Code without table
- " with "
- Time Complexity without table
- " " " with "
- Why overlapping subproblems boon for DP
- Order for multiplying $A_{3\times2}$ $B_{2\times3}$
- General
- 2 ways to multiply $A_{2\times1}$, $B_{1\times2}$ Qs
- which better

Scanned with CamScanner

ABC - # of ways tree
ABCD - # of "
Q: Solve for A1x2 B2x1 C1x4 D4x1
Q: 3 — ways 4 — ways 5 ways
Q: Tree ABCDE
Q: General formula
Q: Same formula also used for ...
Q: 2 very imp things to apply DP
Q: optimal substructure & recursive soln
derive for $P_1 P_2$ — An matrices
Q: Overlapping using tree of 4 matrices A,B,C,D
Q: Time complexity using DP
pg-3 Q: # of repeated problems in
Q: To solve every problem
Q: Total —
Q → 2 approaches
Q: Apply bottom up for
A_1 1x2 A_2 2x1 A_3 1x4 A_4 4x1
and find Parentheses. Also find
a) Time complexity
b) Space "
c) Entries in table.

Q: Algo matrix chain (P)
Q: which approach (out of 2)
Q: Apply on 1-1.x2 B2x1 C1x4 D4x1
Q: What is memoization
Q: which approach
Q: Time complexity
Q: Algorithm
Q: Space complexity
Q: M_1 M_2 M_3 M_4
10x100 100x20 20x5 5x80

Scanned with CamScanner

- Subsequence, substring different
- RAVINDRA → substring & subsequence
- Q: D is a subsequence/not
- → how many possible subsequences Time complexity
- Q: 2 DNA's
  - D_1 = AGCCTCAGT
  - D_2 = GCCT: Find longest common subsequence
- If $D_1 = D_2$ if $D_1 \subset D_2$ if $D_2 \subset D_1$
- Q: Find LCS from RAVINDRA & AJAY
- → Naive way to find
- Time complexity & procedure
- Algo
- → Apply on A-Ravindra B-Ajay
- Why we don't need to find subsequences of B
- Q: Problem
- Optimal substructure & recursive eqn - Approach
- Revise & write recursive eqn
- Q: Recursion tree X = {A, A, A, A} Y = {B, B, B, B}
- Worst case ~~X = {A, A, A, A}~~ tree →
- Best case tree X = {A, A, A, A} Y = {B, B, B, B}
- Q: Draw for (4, 4):
- (3, 3)
- Time & Space
- Q: Overlapping conditions of DP satisfied
- Q: Draw table for 3×3
- Q: for calculating i, what needed already
- Q: Approaches (
- Q: Time, space comp
- Q: Why for top down
- Q: Time, space X = {A, A, A, A} Y = {A, B, A, A}
- Q: Solve X (AAB) Y (ACA) by DP
- Q: —— X (AB (BDAB) Y (B D (ABA)

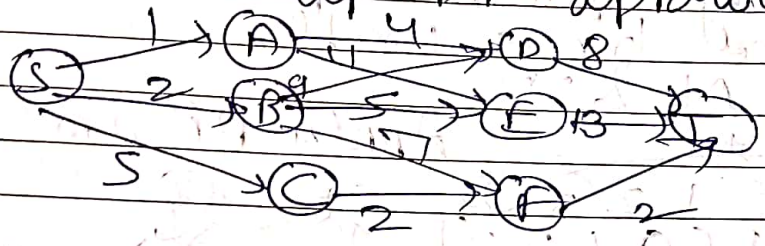
Scanned with CamScanner

Q Find all possible highest matching S.S. from the
Q. Bottom up - Algo
Q) What used if topdown
Q: Stack
Gate 2014 Q: A = q p q r s u
B = p q r s u q u p Find. x + 10y where
x = length of LCS, y = # of LCS possible
Gate 2009 Q: We are given 2 sequences x[m] and y[n]
of length m & n respectively with indices
of x & y starting from 0. l(i,j) is LCS of
x[m] & y[n] is given below

l[i,j] = 0 , if either i=0 & j=0
= expr1 , if i,j>0 & x[i-1] = y[j-1]
= expr2 , if x[i-1] ≠ x[i-1], i,j>0
Find expr1 & expr2

Q-1 l[i,j] is computed by DP using an array
L[M,N] where m=m+1 & n=n+1, st Q
L[i,j] = l(i,j). Find true.
a) All elements of L should be initialized to 0.
b) The values of L(i,j) may be computed in
RMO or CMO
c) The value of L(i,j) can't be computed in
either CMO or RMO
d) L(p,q) needs to be computed before
L(r,s) if either p<r or q<s
Q Multistage graph - 4 properties.
- Q Any)
Q 2 ways to find Multistage - not preferred
Q Greedy - why wrong answers
Q So what we can apply - 2 conditions

- Write recursive eqn
- Draw '' tree
- Tell if DP possible
- Bottom up DP approach using



- Time complexity
- Diff b/w fractional & 0-1 knapsack
- Solve by greedy

obj	1	2	3	Cap = 6
w	1	2	4	
P	10	12	28	

- Tell why greedy fails
- Give an generalize eg to tell why greedy fail
- Brute force time complexity
- For DP → write recursive eqn
- Eg of ICILO and order
- #P of unique subproblems
- Explain table approach
- Time Complexity
- Solve for

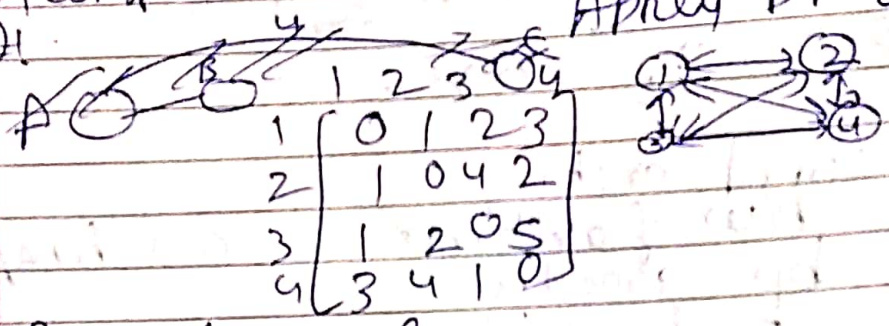
w	1	2	4
P	10	12	28

Cap = 6

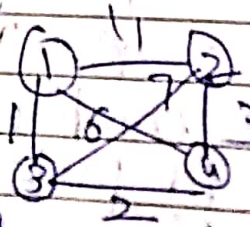
- Time & space complexity
- Eg when DP preferred when brute force
- Algorithm I/P, O/P
- '' Code
- What is Subset sum - General def^n
- why greedy fails
- Brute force
- ...& 6, 2, 3, if W = 5
- DP → Recursive eqn

Scanned with CamScanner

SS (10, 10) tree
Solve for {6, 3, 2, 1}
Time complexity
Travelling salesman Problem-decision
Brute force time complexity
Compare n! & 2^n
Apply DP on



Draw tree also
Draw final graph also
Time complexity by DP.
DP succeed/fail
What type of shortest path we have seen
If we want to find shortest path from each node to every other node
Dijkstra - complexity
Bellman
Both are — algos
Apply DP — Subsequence
Recursive eqn
Apply DP on



Time complexity
Space "
Why
Algo name
" algo(code)

Problems - Flowchart
Draw P, NP chart
Name 3 real life egs
SP →
LP →
What is Euler's tour
Time Complexity
Apply on
Hamiltonian cycle
Time complexity
what is 2CNF
→ Time Comp
→ 3CNF
→ Time Comp
Name & describe 3 optimization problem
Write 3 Decision Problems for them
Relate PP & obf → Time Comp
→ Easy/hard
what problems we consider for NP problems
What is verification algorithm
for Hamiltonian cycle H.B.C.D.P
apply verification algorithm
what is P
NP
why NP not-P is wrong def?
P, NP table
Venn chart
NP, P are
P - easily
NP - "
When any problem A is polynomial time reducible to B con

A. Reduce A to B (if possible)

Q. Given n boolean variables with values $x_1, x_2, x_3 \ldots x_n$ does at least one variable has value "True"?

B. Given n integers $i_1, i_2 \ldots i_n$ is $\max(i_1, i_2 \ldots i_n) > 0$

Q. Both A & B should be —

Q. What is NP-hard

Q. Diagram

Q. What is relation P, NP in and condition

Q. NP-complete

Q. Diagram

Q. When NP = P

Q. When NP ≠ P

Q. If A is NP-hard and A $\xrightarrow{P}$ B then B is

Q. if B is in NP then

Q. Draw a flow chart - Imp

Q. Eg of SAT

Q. SAT is NP —

Q. Eg of 3-CNF sat

Q. What is vertex cover

Q. Eg

2003 Q. Ram & Shyam have been asked to show that a certain problem Π is NP complete. RAM shows a polynomial time reduction from 3-SAT problem to Π, and Shyam shows a polynomial time reduction from Π to 3-SAT what is Π

Q. Let 'S' be an NP complete problem, and 'Q' and 'P' be 2 other problem not known to be in NP. Q is polynomial time reducible to 'S' and 'S' is polynomial time reducible to R. Find P, Q, R, S

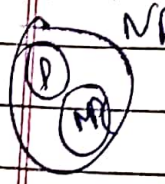
Q) let A be a problem that belongs to the class NP. Then
a) There is no polynomial time algorithm for 'A'
b) If 'A' can be solved deterministically in polynomial time, then P=NP
c) If 'A' is NP-hard, then it is NP-complete
d) A may be undecidable

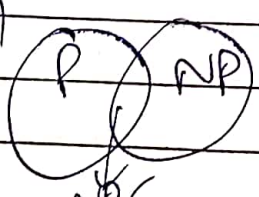
Q) True
a) The problem of determining whether ∃ exists a cycle in an undirected graph is in 'P'
b) The problem of determining whether ∃ a cycle in an undirected graph is 'NP'
c) If a problem A is NP complete ∃ a non deterministic polynomial time algo to solve A.

Q) (Which is not NP-hard.)
a) Hamiltonian circuit problem
b) 0/1 knapsack problem
c) Finding bi-connected components of a graph
d) The graph colouring problem

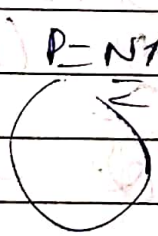
Q) The SHAM3 be a problem of finding hamiltonian cycle in a graph G=(V,E) with V divisible by 3 & DHAM3 be a problem of determining if hamiltonian cycle exists in such graphs.

Q) Suppose a polynomial time algorithm discovered that correctly computes the largest clique in a given graph. In this scenario, which represents correct Venn diagram

a) 

b) 

c) 

d) 

Scanned with CamScanner

SAT extend

Q. Consider DP 2CNF-SAT
a) NP-complete
b) Solvable in polynomial time by reduction to directed graph
c) Solvable in const time since any I/P instance is satisfied
d) NP-hard; but not NPC

Q. If an NP hard problem can be solved in poly time then P=NP (T/F)

Q) Why we focus more on NP complete than NP-hard - 3

Q) The set of NP problems is not closed under polynomial time reduction (T/F)
bearing